

Paso 6 - Repetir con while | Solucionario

Las cinco soluciones del paso 6. Casi todas muestran **dos ejecuciones**: la normal y la del caso borde, que es donde los ciclos fallan.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso06` y `javac *.java`.

B6.1 - La suma con centinela

El patrón centinela: se lee **antes** del `while`, y al final del cuerpo se vuelve a leer. El `0` no se suma: es la señal de parar.

▼ Ver la solución (B6_1_Centinel.java)

B6_1_Centinel.java

```
import java.util.Scanner;

public class B6_1_Centinel {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int suma = 0;
        int cantidad = 0;

        System.out.print("Numero (0 para terminar): ");
        int numero = sc.nextInt();

        while (numero != 0) {
            suma += numero;
            cantidad++;
            System.out.print("Numero (0 para terminar): ");
            numero = sc.nextInt();
        }

        System.out.println("Ingresaste " + cantidad + " numeros");
        System.out.println("Suma: " + suma);

        if (cantidad == 0) {
            System.out.println("Promedio: no hay numeros que promediar");
        } else {
            System.out.println("Promedio: " + (double) suma / cantidad);
        }

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Numero (0 para terminar): 10
Numero (0 para terminar): 25
Numero (0 para terminar): 7
Numero (0 para terminar): 0
Ingresaste 3 numeros
Suma: 42
Promedio: 14.0

--- ejecucion 2
Numero (0 para terminar): 0
Ingresaste 0 numeros
Suma: 0
Promedio: no hay numeros que promediar
```

Lo que importa. La lectura aparece **dos veces**: una antes del `while` y otra al final del cuerpo. Es incómodo pero necesario: el `while` necesita algo que comparar **antes** de la primera vuelta. (En el paso 11, `Validador.leerEntero` esconde esta doble lectura dentro de un método, y por eso se ve tan limpio.)

El caso del 0 al principio es el que pedía el enunciado: sin el `if (cantidad == 0)`, `suma / cantidad` es `0 / 0`. Con enteros eso lanza `ArithmeticException: / by zero`; con el cast a `double` no revienta, pero imprime `NaN`, que es igual de inútil. Las dos formas de fallar aparecen en la demo 10.

El `(double) suma / cantidad` es el cast del paso 3: sin él, `42 / 3` daría 14 y no 14.0, y con `43 / 3` daría 14 en vez de 14.33.

B6.2 - Adivina el número

Otro centinela, pero la condición no es un valor fijo sino una comparación.

▼ Ver la solución (B6_2_Adivina.java)

B6_2_Adivina.java

```
import java.util.Scanner;

public class B6_2_Adivina {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int secreto = 7;
        int intentos = 0;

        System.out.print("Adivina (1 a 10): ");
        int intento = sc.nextInt();
        intentos++;

        while (intento != secreto) {
            if (intento < secreto) {
                System.out.println("Mas alto");
            } else {
                System.out.println("Mas bajo");
            }
            System.out.print("Adivina (1 a 10): ");
            intento = sc.nextInt();
            intentos++;
        }

        System.out.println("Acertaste en " + intentos + " intentos!");
        sc.close();
    }
}
```

Salida:

```
Adivina (1 a 10): 3
Mas alto
Adivina (1 a 10): 9
Mas bajo
Adivina (1 a 10): 7
Acertaste en 3 intentos!
```

Lo que importa. El `intentos++` va en los **dos** lugares donde se lee, o el conteo queda corto. Es el error más común del ejercicio.

Los dos extras. El secreto al azar es una línea:

```
int secreto = (int) (Math.random() * 10) + 1;
```

`Math.random()` da un decimal entre 0 y 1 (sin llegar al 1); multiplicado por 10 queda entre 0 y 9.99; el cast lo corta a 0-9, y el `+ 1` lo mueve a 1-10. Para limitar a tres intentos, la condición del `while` lleva **dos** cosas, y al final hay que preguntar por qué salió:

```
while (intento != secreto && intentos < 3) {  
    ...  
}  
  
if (intento == secreto) {  
    System.out.println("Acertaste en " + intentos + " intentos!");  
} else {  
    System.out.println("Perdiste, era el " + secreto);  
}
```

Ese `if` de después del ciclo es un patrón que vale la pena reconocer: cuando un `while` puede terminar por **dos** razones distintas, casi siempre hay que preguntar cuál fue.

B6.3 - El menú que no se rinde

`do-while` por fuera, `switch` por dentro. El menú se muestra antes de preguntar, y por eso no sirve un `while` normal.

▼ Ver la solución (B6_3_Menu.java)

B6_3_Menu.java

```
import java.util.Scanner;

public class B6_3_Menu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println();
            System.out.println("1) Saludar");
            System.out.println("2) Sumar dos numeros");
            System.out.println("3) Contar hasta 10");
            System.out.println("0) Salir");
            System.out.print("Opcion: ");
            opcion = sc.nextInt();

            switch (opcion) {
                case 1:
                    System.out.println("Hola!");
                    break;
                case 2:
                    System.out.print("Primer numero: ");
                    int a = sc.nextInt();
                    System.out.print("Segundo numero: ");
                    int b = sc.nextInt();
                    System.out.println(a + " + " + b + " = " + (a + b));
                    break;
                case 3:
                    for (int i = 1; i <= 10; i++) {
                        System.out.println(i);
                    }
                    break;
                case 0:
                    System.out.println("Chao");
                    break;
                default:
                    System.out.println("Esa opcion no existe");
            }
        } while (opcion != 0);

        sc.close();
    }
}
```

```
}  
}
```

Salida:

```
1) Saludar  
2) Sumar dos numeros  
3) Contar hasta 10  
0) Salir  
Opcion: 1  
Hola!  
  
1) Saludar  
2) Sumar dos numeros  
3) Contar hasta 10  
0) Salir  
Opcion: 2  
Primer numero: 5  
Segundo numero: 7  
5 + 7 = 12  
  
1) Saludar  
2) Sumar dos numeros  
3) Contar hasta 10  
0) Salir  
Opcion: 9  
Esa opcion no existe  
  
1) Saludar  
2) Sumar dos numeros  
3) Contar hasta 10  
0) Salir  
Opcion: 0  
Chao
```

Lo que importa. `int opcion;` se declara **antes** del `do`, porque la condición del final tiene que verla. Y el `default` no lleva `break` porque es el último: igual conviene acostumbrarse a ponerlo, porque el día que agregues un `case` abajo, el `default` se va a seguir de largo hacia él.

El caso 3 usa un `for` aunque estemos en el paso del `while`: son 10 vueltas conocidas, y el paso 7 explica por qué ahí el `for` es la herramienta correcta.

Prueba a escribirle `abc` a la opción: el menú muere con `InputMismatchException`. Ese es exactamente el ejercicio **B11.3**, donde el mismo menú se vuelve indestructible cambiando una línea.

B6.4 - El validador insistente

El `while` de validación del paso 6, más un contador.

▼ Ver la solución (B6_4_ValidarNota.java)

B6_4_ValidarNota.java

```
import java.util.Scanner;

public class B6_4_ValidarNota {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int errores = 0;

        System.out.print("Nota (1.0 a 7.0): ");
        double nota = sc.nextDouble();

        while (nota < 1.0 || nota > 7.0) {
            errores++;
            System.out.print("Nota invalida. Otra vez: ");
            nota = sc.nextDouble();
        }

        System.out.println("Nota aceptada: " + nota + " (te equivocaste " +
            errores + " veces)");
        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Nota (1.0 a 7.0): 9
Nota invalida. Otra vez: -2
Nota invalida. Otra vez: 5.5
Nota aceptada: 5.5 (te equivocaste 2 veces)

--- ejecucion 2
Nota (1.0 a 7.0): 5.5
Nota aceptada: 5.5 (te equivocaste 0 veces)
```

Lo que importa. El `||` y no `&&`: la nota está mala si es menor que 1.0 o mayor que 7.0. Con `&&` la condición no se cumple nunca (ningún número es a la vez menor que 1 y mayor que 7), así que el `while` no se ejecutaría jamás y aceptarías cualquier cosa. Es el error más frecuente del ejercicio, y **no da ningún síntoma** hasta que alguien escribe un 9.

La segunda ejecución muestra el caso que se olvida: si la nota viene buena a la primera, el `while` no entra ni una vez y el contador queda en 0.

B6.5 - Desafío: dar vuelta un número

El mismo par `/ 10` y `% 10` del B3.3, ahora dentro de un `while` que no sabe cuántos dígitos vienen.

▼ Ver la solución (B6_5_AlReves.java)

B6_5_AlReves.java

```
import java.util.Scanner;

public class B6_5_AlReves {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Numero: ");
        int numero = sc.nextInt();

        int digitos = 0;
        int suma = 0;

        System.out.print("Al revés: ");
        while (numero > 0) {
            int ultimo = numero % 10;
            System.out.print(ultimo);
            suma += ultimo;
            digitos++;
            numero = numero / 10;
        }
        System.out.println();

        System.out.println("Tiene " + digitos + " digitos");
        System.out.println("Suma de sus digitos: " + suma);

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Numero: 5730
Al reves: 0375
Tiene 4 digitos
Suma de sus digitos: 15

--- ejecucion 2
Numero: 8
Al reves: 8
Tiene 1 digitos
Suma de sus digitos: 8
```

Lo que importa. Aquí el `while` es obligatorio: **no sabes** cuántos dígitos tiene el número hasta que lo desarmas. Es el ejemplo más claro de la diferencia con el `for` del paso 7.

Fíjate en que `numero` se va **destruyendo** en el proceso: al terminar el ciclo vale 0. Si necesitaras el original después (para compararlo con el invertido, por ejemplo, y ver si es capicúa), habría que guardarlo en otra variable antes de empezar.

El `0375` del enunciado se ve raro pero es correcto: 5730 al revés es 0375, con el cero adelante. Como lo imprimimos **dígito por dígito** con `print`, el cero se ve. Si en vez de imprimirlos armaras un número (`invertido = invertido * 10 + ultimo`), el cero de la izquierda se perdería y quedaría 375.

Volver a la [hoja del paso 6](#) o al [paso 6 de la guía](#).